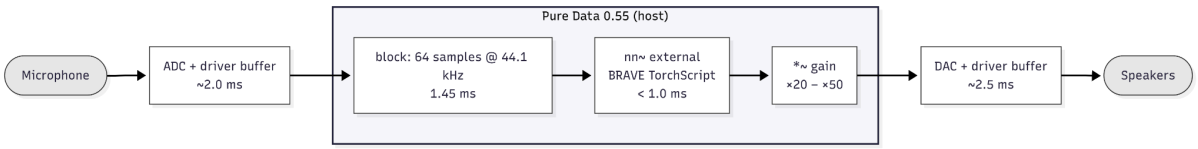
Chapter 3: Methodology

In this chapter we thoroughly describe the technical setup, various training routines as well as the evaluation mechanisms that have been used to explore the research question of this thesis: is it possible for a causal streaming neural autoencoder to perform a voice-driven instrument resynthesis that is perceptually very close to the original one with a latency of less than 10 ms, and what kinds of training instabilities and failure modes are revealed when this task is performed with heterogeneous, limited-size datasets? The present research methodology revolves around four main units system architecture, training paradigm, dataset preparation, and diagnostic protocols each of which is associated with its respective evaluation done in Chapter 5.

3.1 System Architecture

The real-time resynthesis pipeline acts as a low-latency end-to-end stream, from acoustic input to digital output. The signal path is a straight line: an acoustic input is taken by a microphone, then through an ADC, processed by the BRAVE model in a host environment, and converted back to analog via a DAC.



[FIGURE 3.1: Block diagram of the real-time pipeline showing Mic → ADC → Pure Data (nn~ + BRAVE) → Gain → DAC → Speakers]

One primary design limitation is that the system must provide a latency of less than 10 ms from the start to the end of the process. According to Wessel and Wright (2002), any delay in this range might already be considered harmful to the user's intimate musical control experience. To implement this constraint, a combination of the Pure Data (Pd) and the nn~ external (Caillon & Esling, 2021) was chosen.

Design Rationale: Host Environment. The selection of Pure Data and nn~ was finalized subsequent to a comparison with another option of building a bespoke C++ audio engine based on the Torch C++ API (Libtorch). While a custom engine may provide a greater capacity for optimisation, its development would have required a significant amount of time creating the mundane audio I/O and interface management. Besides being the target for many deployments of RAVE-family models (Caillon & Esling, 2021), Pure Data was also chosen due to its stability and ability to immediately include standard DSP operations such as gain multipliers and limiters, which turned out to be quite essential considering the low raw output level of the initial model iterations.

Architectural Backbone: Streaming-Causal RAVE. The synthesis engine utilized by the system is called BRAVE (Bravely Realtime Audio Variational autoEncoder). Where normal RAVE employs non-causal convolutions and experiences a delay in buffering of about 50 ms (Caillon & Esling, 2021), BRAVE is constructed to have a streaming-causal layout (Caspe et al., 2025). This is done by limiting the model's temporal context to only the current and previous samples, so latency is almost solely determined by the size of the processing block and the cost of a model inference. Table 3.1 breaks down the estimated end-to-end latency into its constituent contributions.

Table 3.1. Estimated latency decomposition for the deployed pipeline (block size 64 samples at 44.1 kHz, host environment Pure Data 0.55 on Windows 11 with the WASAPI driver). Block-size and model-forward values are calculated analytically based on the configuration parameters; the driver-buffer values depend on the WASAPI configuration and have been observed through informal live testing rather than measuring by a calibrated loopback test. Performing a formal loopback-based latency measurement is recognized as the future work in Chapter 6.

Component	Approximate latency
Audio driver buffer (input)	~2.0 ms
ADC + Pure Data block (64 samples @ 44.1 kHz)	1.45 ms
BRAVE model forward pass (per block, RTX 5080)	< 1.0 ms
Pure Data output buffer + DAC	~2.5 ms
Total end-to-end (estimated)	**~7 ms**

Estimated latency decomposition for the deployed pipeline (block size 64 samples at 44.1 kHz, host environment Pure Data 0.55 on Windows 11 with the WASAPI driver). Block-size and model-forward values are computed analytically from configuration parameters; driver-buffer values are reported by the WASAPI configuration and observed informally during live testing rather than measured through a calibrated loopback test. Formal loopback-based latency measurement is identified as future work in Chapter 6.

The 7 ms total latency estimate is consistent with the latency range that Wessel and Wright (2002) approximate.

After establishing the architectural constraints of the runtime system, the next section outlines the training approach to obtaining stable and accurate instrument-like generative behaviour within these constraints.

3.2 Training Paradigm

Experimental Premise: Unpaired Cross-Modal Resynthesis. It is first of all necessary to make the point that the system is not trained on paired voice-to-instrument examples before introducing two-phase training. BRAVE is essentially an instrument-domain autoencoder, trained with only guitar or drum recordings; the voice is a new source of input only at the time of inference, being suggested that the encoder's learnt latent manifold will demonstrate the generalisation to vocal excitation signals out-of-distribution (OOD). This suggestion is preceded by the fact that neural audio autoencoders go by the organisation of neural latent spaces mainly spectral-temporal structure rather than source-specific categories. Playing with an empirical inquiry whether this the case of guitar and drums, and the rest that emerge when it fails, will be one of the main parts of this thesis's contribution.

The training procedure is broken down into two phases that separate the learning of a compact latent representation from the synthesis of perceptually plausible waveforms.

Phase 1: Representation Learning. To begin with, the model serves as a conventional Variational Autoencoder (Kingma & Welling, 2013). The encoder and decoder are being trained at the same time to minimise a multi-scale Short-Time Fourier Transform (STFT) reconstruction loss that refers to magnitude spectrograms of input and output at multiple temporal resolutions (Engel et al., 2020; Caillon & Esling, 2021). The spectral loss stems from perceptual considerations: subtle phase variations are largely inaudible to human listeners, so the model is not penalised for them. Latent space is regularised via a Kullback-Leibler (KL) divergence term that pushes the posterior distribution toward a standard Gaussian prior (Kingma & Welling, 2013).

Phase 2: Adversarial Fine-Tuning. At the point where reconstruction loss settles, the encoder is locked and the decoder is fine-tuned to become the generator part of a Generative Adversarial Network (GAN; Goodfellow et al., 2014). Consistent with use of RAVE (Caillon & Esling, 2021), the encoder is frozen so that the latent representation learnt during Phase 1 cannot be altered and destabilised by adversarial gradients (i.e., the bottleneck structure is kept intact). A discriminator network differentiates between genuine audio samples and audios reconstructed by the decoder. According to the GAN-vocoder literature, the adversarial objective of a GAN is one of recovering high-frequency textural detail and transient information, which is generally smoothed over by purely spectral losses (Kong et al., 2020).

Design Rationale: Beta Scheduling

In the first training iteration (guitar_v1), a constant beta of 0.1 was applied from step zero, as inherited from the unmodified c128_r10.gin configuration shipped with the BRAVE repository. This configuration resulted in posterior collapse, a well-documented training instability in which the latent space becomes uninformative because KL pressure is applied before the model has learned a meaningful reconstruction (Bowman et al., 2016).

Theoretical motivation. The best-known VAE literature stabilisation technique is to anneal the KL term slowly and in doing so, the encoder first learns salient input features and only then when forced compresses feature towards the prior (Bowman et al., 2016).

Adopted solution. All future runs (v2-v6) utilize a log-space warmup such that the very first beta value is 1×10^{-4} which means, practically no pressure and then the beta value increases exponentially to 0.1 during the first one million steps.

Observed outcome. No posterior collapse was seen in the following runs with the warmup schedule, and Phase 1 produced stable rising KL curves in all five iterations, as noted in Chapter 5.

Design Rationale: Latent Dimensionality

The latent size was originally set to 128, which is the default size for RAVE. Results from the guitar_v2 diagnostics indicated a very low usage of 0.005 nats per dimension, which means the latent space was much bigger than what the dataset could fill. Further, the latent dimension has been decreased to 16, which is the value that matches established practice in the RAVE community for guitar-scale datasets. The smaller size then seems to trigger more effective latent use on the subsequent versions where the utilization went up to around 0.04 nats per dimension.

Training Configuration

In all guitar models, Adam optimizer is used with RAVE default learning rate, and the beta schedule is specified separately in the per iteration.gin file. The training batch size remains constant at 8. Phase 1 covers 1M steps in v1-v4 iterations and it is further lengthened to 1.5 M in v5 and v6 so that the longer reconstruction phase can be given before the start of adversarial training. Each complete execution anticipates about 3 M total steps.

Hardware Environment

The training was conducted on a single workstation with the following specifications: NVIDIA RTX 5080 GPU (16 GB VRAM), Windows 11, Python 3.14, PyTorch 2.11.0+cu128, acids-rave 2.3.1, pytorch-lightning 2.6.1. At the rate of 9-12 optimization steps per second, it takes about 90 to 120 wall-clock hours for each full run.

Checkpoint Selection

It is a common practice to create checkpoints after every 10 000 training steps. RAVE's own best.ckpt feature automatically records the minimum validation reconstruction loss for each run segment. In case of iterations where the model from the last epoch had deteriorated after Phase 2 (the most typical being guitar_v6), the usual checkpoint was chosen manually by listening comparison of different intermediate epochs, as explained in Chapter 5.

Reproducibility. All the details of the experiments including configuration files, preprocessing scripts, and evaluation utilities were version-controlled during the whole development phase and will be made public via the thesis repository on Git, along with the diagnostic Python scripts and the Pure Data deployment patches.

3.3 Datasets and Preprocessing

The effectiveness of RAVE-family model hinges largely on the amount, diversity, and similarity of the training data. Here we list the data used, how we chose them, and the steps we took to get them ready for training. Although the dataset scale here is fairly small by the standards of generative audio, it is still in line with realistic availability of specific instrument open datasets, particularly for guitar.

Guitar Corpus. The guitar corpus is a mix of three publicly available sources filtered for compatible audio formats. The three sources complement each other: GuitarSet offers mic'd acoustic recordings with great performance variation, Guitar-TECHS provides clean electric direct-input (DI) signals free from amplifier and room colouring, and IDMT-SMT-Guitar dataset 4 offers timbral diversity across different electric and acoustic guitars and capture paths. The mix was aimed at broadening the model's guitar timbre familiarisation while tolerating some variability in noise floor and spectral character. The impact of this choice is measured in the noise-floor analysis below and the topic continues in Chapter 5.

Table 3.2. Breakdown of the guitar training corpus (mono, 16-bit PCM, 44.1 kHz) after format filtering. The IDMT-SMT-Guitar dataset 2 was not included because its 24-bit files were incompatible with the LMDB construction phase, as explained in the preprocessing pipeline section. Once preprocessed, the LMDB generated shows n_seconds = 57665 (~16 h), a larger number which is due to overlapping segmentation rather than additional source audio.

Source	Recording method	Raw duration	File count
GuitarSet (Xi et al., 2018)	Mic'd acoustic, both mic positions	~3.05 h	360
Guitar-TECHS DI channel (Pedroza et al., 2024)	Electric DI	~1.5 h	32 (post stereo filtering)
IDMT-SMT-Guitar dataset 4 (Kehling et al., 2014)	Mixed: acoustic_mic, acoustic_pickup, electric DI	~4.35 h	507
Total raw	—	**~9 h**	**899**

Drums Corpus. The drums section is composed from the MIDI Dataset of Groove (Gillick et al., 2019) consisting of approximately 10.86 hours total after stereo-to-mono conversion. The decision to utilize Groove comes from two considerations. Firstly, it covers a dataset scale much bigger than those RAVE and BRAVE

demo datasets used in the past, strongly minimising risks from dataset-size-induced training instabilities. Secondly, by having a wide range of tempos and rhythmic patterns performed by several professional drummers, the Groove set-up is expected to enhance the learned latent representation's robustness, which is critical for vocal-to-drums cases with transient-rich percussive events.

Preprocessing Pipeline. A five-stage Python pipeline was realized with librosa and soundfile:

1. **Validation.** Each input file is loaded with librosa.load. Files that fail to load or contain no audio are discarded.
2. **Mono conversion.** Stereo files are downmixed to a single channel.
3. **Standardisation.** Audio is resampled to 44.1 kHz and saved as 16-bit PCM WAV.
4. **Artefact filtering.** Metadata artefacts (notably __MACOSX/.wav files inside Mac-archived datasets) and 24-bit files are removed. A preprocessing validation stage was incorporated after empirical observation that incompatible 24-bit audio files caused instability in the standard rave preprocess pipeline.
5. **LMDB construction.** We use rave preprocess --no-lazy to pre-decode the processed WAV files into a Lightning Memory-Mapped Database (LMDB). This reduces disk I/O overhead during training.

Noise-Floor Analysis. In order to characterise the heterogeneity of the guitar corpus, a noise-floor analysis was conducted. The Root Mean Square (RMS) energy of each file was calculated in non-overlapping 100 ms windows, and the 5th percentile of these RMS values was used as the noise-floor estimate, which was then converted to dBFS. Per-source aggregates (median and inter-quartile range) give an idea of the spread in the corpus. The complete results, which are shown in Chapter 5, reveal a 35.8 dB range between the cleanest electric DI sources and the loudest microphone-captured acoustic sources, and they contribute to the interpretation of latent-space behaviour later in the thesis.

Task-Specific Difficulty Considerations. In addition to the composition of the datasets, the two case studies also differ in intrinsic task difficulties that influence the analysis presented in Chapter 5. Guitar resynthesis, when compared to drum resynthesis, is more challenging in terms of latent continuity and harmonic precision because guitar signals are sustained, pitch-structured, and mainly consist of stable harmonic relationships, micro-dynamic articulations, and long-duration resonances. Percussive audio, on the other hand, is characterized by transient events and broader spectral envelopes, which can tolerate small latent perturbations more easily. Therefore, dataset heterogeneity, latent under-utilization, and adversarial instability are likely to show up more strongly in the guitar case, both because the target signal requires more detailed representational fidelity and because the cross-modal mapping from voice to guitar involves a larger acoustic gap than voice to drums.

3.4 Diagnostic Methodology

In this thesis, failure modes are defined as the systematic departures that prevent stable or perceptually meaningful resynthesis. To make the analysis more understandable, these failures are identified as two partially overlapping groups: training instabilities that correspond to optimisation-dynamic failures that can be seen during training (posterior collapse and discriminator dominance), and degenerate behaviours that refer to pathological output characteristics that can be seen after training (latent underutilisation and output amplitude degeneration). The diagnostic framework that is offered by this section aims at detecting both groups: in-training diagnostics that are monitored continuously through the TensorBoard, post-training evaluation metrics that are calculated on each candidate checkpoint, and reconstruction validation on in-distribution audio.

In-Training Diagnostics. Five main TensorBoard scalars are recorded during training

- regularization (KL divergence): During Phase 1, if the values go up, the latent-space utilisation is effective. Going towards zero indicates posterior collapse.
- multiband_spectral_distance: the main measuring tool for reconstruction. It is expected to go down steadily during Phase 1 and to recover after the brief spike at Phase 2 onset.

- pred_real and pred_fake: are the discriminator's logits on real and generated audio correspondingly. According to the present experiments, a difference of less than 3 between pred_real and pred_fake was a sign of relatively stable adversarial behaviour, whereas values greater than 5 were quite often a sign of discriminator dominance and degraded outputs, a failure pattern consistent with the mode-collapse dynamics described by Goodfellow et al. (2014).
- loss_dis: overall discriminator loss, which is the main concern to establish whether the discriminator has completely collapsed or not.

Post-Training Evaluation Metrics. For each candidate checkpoint, a Python script is executed for the evaluation of a synthetic-input behavior using five spectrally distinct inputs: silence; sine waves at 220, 440, and 880 Hz; a linear frequency sweep from 100 to 2000 Hz; and white Gaussian noise.

The following criteria are then considered:

- **Encoder differentiation**. The mean pairwise absolute difference between latent vectors for different inputs should be higher than 0.5. To set this threshold, we did preliminary testing: models that were completely collapsed or very weakly responsive to inputs gave pairwise latent differences below 0.2, while functionally responsive checkpoints gave values that were always above 0.8.
- **Decoder responsiveness.** The pairwise mean absolute difference between audio outputs should be higher than 0.01, a value that was also based on empirical observations; even in the most degenerate cases the pairwise output difference was below 0.005, which corresponded to perceptually indistinguishable outputs.
- **Output amplitude.** The mean absolute amplitude for tonal inputs needs to be within [0.05, 0.3] to ensure the model generates a clearly audible signal without any type of clipping.

Reconstruction Validation.A final diagnostic runs the trained autoencoder on guitar audio from GuitarSet that is in distribution. Two metrics are derived: the input/output amplitude ratio (want it close to 1.0) and the magnitude-STFT correlation of the input vs. output. As there is no commonly agreed upon "perceptually acceptable" level of neural audio reconstruction, when expressed as STFT magnitude correlation, in the literature, this thesis uses a criterion greater than 0.3 as an empirical heuristic, based on informal listening tests against checkpoints with lower correlations.
This decision is considered a practical operating threshold rather than a normative statement, and the drawbacks of it are discussed in Chapter 6.

Validity Limitations of the Diagnostic Framework. Four caveats apply to the framework presented above and are acknowledged here rather than relegated to the conclusions.

Firstly, synthetic sine-wave probing is a structural test of input differentiation and does not in itself establish perceptual realism of the output; it only verifies that the network's internal mappings are non-degenerate.

Secondly, magnitude-STFT correlation, which is used for checking reconstruction, only measures spectral envelope similarity. It doesn't take into account phase-dependent effects, micro-temporal modulations or the features that can influence perceptual judgement of audio quality. Hence Chapter 5 listening evaluation serves as a complement, not a substitute, to this quantitative measure.

Thirdly, the [0.05, 0.3] amplitude band used as a sanity check is an engineering heuristic chosen to flag clipping and silent-output failures; it is not derived from a perceptual model of loudness.

Fourthly, due to the exploratory and iterative nature of the experiments and also because many intermediate models failed basic reconstruction criteria, no formal perceptual evaluation (such as MUSHRA or MOS testing)

was conducted. Given the large number of failed intermediate checkpoints, formal perceptual testing was considered premature until basic reconstruction and responsiveness criteria had first been satisfied. Instead, listening evaluation was done informally by the author at each checkpoint.

Together, these protocols enable each training iteration in Chapter 5 to be assessed against stable quantitative criteria, at the same time being open about what those criteria measure and what they do not. The implementation details that bring this methodology to work, software environment, compatibility patches, training scripts, and Pure Data deployment patches, are conveyed in the next chapter.

References

Caillon, A., & Esling, P. (2021). RAVE: A variational autoencoder for fast and high-quality neural audio synthesis. *arXiv preprint arXiv:2111.05011*.

Caspe, F., Shier, J., Sandler, M., Saitis, C., & McPherson, A. (2025). Designing neural synthesizers for low-latency interaction. *arXiv preprint arXiv:2503.11562*.

Bowman, S. R., Vilnis, L., Vinyals, O., Dai, A. M., Jozefowicz, R., & Bengio, S. (2016). Generating sentences from a continuous space. *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, 10–21. https://doi.org/10.18653/v1/K16-1002

Engel, J., Hantrakul, L., Gu, C., & Roberts, A. (2020). DDSP: Differentiable digital signal processing. *Proceedings of the 8th International Conference on Learning Representations (ICLR)*.

Gillick, J., Roberts, A., Engel, J., Eck, D., & Bamman, D. (2019). Learning to groove with inverse sequence transformations. *Proceedings of the 36th International Conference on Machine Learning (ICML)*.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27.

Kehling, C., Abeßer, J., Dittmar, C., & Schuller, G. (2014). Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters. *Proceedings of the 17th International Conference on Digital Audio Effects (DAFx)*.

Kingma, D. P., & Welling, M. (2013). Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*.

Kong, J., Kim, J., & Bae, J. (2020). HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. *Advances in Neural Information Processing Systems*, 33, 17022–17033.

Pedroza, H., Abreu, W., Corey, R. M., & Roman, I. R. (2024). Guitar-TECHS: An electric guitar dataset covering techniques, musical excerpts, chords and scales using a diverse array of hardware. *Proceedings of the 27th International Conference on Digital Audio Effects (DAFx)*.

Wessel, D., & Wright, M. (2002). Problems and prospects for intimate musical control of computers. *Computer Music Journal*, *26*(3), 11–22. https://doi.org/10.1162/014892602320582945

Xi, Q., Bittner, R. M., Pauwels, J., Ye, X., & Bello, J. P. (2018). GuitarSet: A dataset for guitar transcription. *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*